

Problem Set 2

Fundamentals of Numerical Programming I

LEONARD STORCKS

January 6, 2026

Problem 1. Tridiagonal solver

In this task we want to practice the principle of Gaussian elimination on a simplified setup. Consider the invertible tridiagonal matrix

$$\underline{\underline{A}} = \begin{pmatrix} d_0 & u_0 & 0 & \dots & 0 & 0 \\ l_1 & d_1 & u_1 & 0 & \dots & \\ 0 & l_2 & d_2 & u_2 & 0 & \dots \\ 0 & 0 & \ddots & \ddots & \ddots & 0 \\ 0 & \dots & & l_{N-2} & d_{N-2} & u_{N-2} \\ 0 & 0 & \dots & 0 & l_{N-1} & d_{N-1} \end{pmatrix} \quad (1)$$

(by convention $l_0 = 0, u_{N-1} = 0$) where we want to solve

$$\underline{\underline{A}}\underline{x} = \underline{b} \quad (2)$$

for $\underline{x}$.

- (a) Write the system in the form $(\underline{\underline{A}}|\underline{b})$. Find the row operations which eliminate the lower diagonal, starting with l_1 . *Hint*: You start by subtracting a multiple of the first row from the second such that $l_1 = 0$ and continue downward.
- (b) Once the lower diagonal is eliminated, the system can be solved bottom-up. Given the transformed system with eliminated lower diagonal, find an expression for the solution vector. *Hint*: Start by finding an expression for the last entry x_{N-1} in $\underline{x} = (x_0, x_1, \dots, x_{N-2}, x_{N-1})^T$, continue with x_{N-2} and so on.
- (c) Implement the tridiagonal solver in Python. The signature of the function as well as the implementation of a matrix-vector product are given in Code-Snippets 1 to 3. What

```

1      """
2      The tridiagonal matrix is given by
3      [u_0, u_{N-2}], [d_0, d_{N-1}], [l_1, l_{N-1}].
4      By convention all arrays are of size N, with
5      u_{N-1} = 0, l_0 = 0. This way the
6      (l_i, d_i, u_i) are the values
7      from the i-th row.
8      """
9
10     import numpy as np
11     import matplotlib.pyplot as plt
12     from scipy.linalg import lu_factor, lu_solve
13
14     def tri_vector_product(l, d, u, x):
15         """
16         Compute Ax where A is a tridiagonal
17         matrix defined by l, d, u.
18         """
19         return x * d + np.roll(x, 1) * l + np.roll(x, -1) * u
20
21     def tri_to_matrix(l, d, u):
22         """
23         Convert a tridiagonal matrix to a
24         full matrix.
25         """
26         N = d.shape[0]
27         return np.diag(d) + np.diag(l[1:], -1) + np.diag(u[:-1], 1)

```

Code-Snippet 1: Python code for a matrix-vector product $\underline{\underline{A}}\underline{\underline{x}}$ where $\underline{\underline{A}}$ is tridiagonal and signature for a tridiagonal solve of $\underline{\underline{A}}\underline{\underline{x}} = \underline{\underline{b}}$. Continued in Code-Snippet 2.

is the scaling of your algorithm? Compare to $\mathcal{O}(N^3)$ of standard LU factorization.

```

1      def tri_solve(l, d, u, b):
2          """
3              Solve  $Ax = b$  where  $A$  is a tridiagonal
4              matrix defined by  $l$ ,  $d$ ,  $u$ .
5
6              Args:
7                   $l$ ,  $d$ ,  $u$ : arrays of size  $N$ 
8                   $b$ : array of size  $N$ 
9
10             Returns:
11                 Solution to  $Ax = b$  of size  $N$ 
12             """
13             N = diag.shape[0]
14
15             # backward elimination of
16             # the lower diagonal
17             for i in range(0, N - 1):
18                 # TODO
19
20             # forward substitution
21             x = np.zeros_like(diag)
22             x[-1] = # TODO
23
24             for i in range(N - 2, -1, -1):
25                 # TODO
26
27             return x

```

Code-Snippet 2: Continuation of Code-Snippet 1. Continued in Code-Snippet 3

```

1      # test setup
2      l = np.array([0.0, 2.0, 4.0, 6.0, 2.0, 6.0])
3      d = np.array([7.0, 9.0, 10.0, 15.0, 9.0, 7.0])
4      u = np.array([4.0, 6.0, 5.0, 8.0, 7.0, 0.0])
5
6      # predefined solution
7      x = np.array([5.0, 7.0, 8.0, 9.0, 5.0, 1.0])
8
9      # tridiagonal representation to full matrix
10     A = tri_to_matrix(l, d, u)
11
12     # matrix vector product
13     assert np.allclose(A @ x, tri_vector_product(l, d, u, x))
14     b = tri_vector_product(l, d, u, x)
15
16     # tridiagonal solve
17     x_tilde = tri_solve(l, d, u, b)
18     assert np.allclose(x, x_tilde)

```

Code-Snippet 3: Continuation of Code-Snippet 2.

Problem 2. Jacobi iteration

Consider random strictly diagonally dominant matrices $\underline{\underline{R}}_N$ with

$$\forall i : |r_{ii}| > \sum_{j \neq i} |a_{ij}| \quad (3)$$

These can be generated by

- (a) Draw a random Gaussian matrix $\underline{\underline{G}}_N \in \mathbb{R}^{n \times n}$, $G_{ij} \sim \mathcal{N}(\mu = 0, \sigma^2 = 1)$.
- (b) Enforce strict diagonal dominance by

$$R_{ij} = \begin{cases} G_{ij}, & i \neq j, \\ \delta + \sum_{j \neq i} |G_{ij}|, & i = j \end{cases} \quad (4)$$

with margin $\delta = 0.1$.

A Gaussian RHS in $\underline{\underline{A}}\underline{x} = \underline{b}$, $b_i \sim \mathcal{N}(\mu = 0, \sigma^2 = 1)$, $i = 1, \dots, N$ is used throughout this task.

Jacobi iteration: For Jacobi iteration, we start with the trivial decomposition

$$\underline{\underline{A}} = \underline{\underline{D}} - (\underline{\underline{L}} + \underline{\underline{U}}), \quad \text{diagonal part } \underline{\underline{D}}, \quad \text{negative left-below-diagonal part } \underline{\underline{L}} \\ \text{negative right-above-diagonal part } \underline{\underline{U}} \quad (5)$$

and iterate

$$\underline{x}^{(n+1)} = \underline{\underline{D}}^{-1}\underline{b} + \underbrace{\underline{\underline{D}}^{-1}(\underline{\underline{L}} + \underline{\underline{U}})}_{=: \underline{\underline{M}}} \underline{x}^{(n)}, \quad (\underline{\underline{D}}^{-1})_{ii} = \frac{1}{A_{ii}} \quad (6)$$

You can start the iteration with $\underline{x}^{(0)} = \underline{0}$.

- (a) Show that Jacobi iteration converges for a strictly diagonally dominant matrix, i.e. the spectral radius $\rho_s(\underline{\underline{M}}) < 1$.
- (b) Implement the generation of $\underline{\underline{R}}_N$ and $\underline{b} \in \mathbb{R}^N$ with $b_i \sim \mathcal{N}(\mu = 0, \sigma^2 = 1)$, $i = 1, \dots, N$ in Python.
- (c) Implement the Jacobi iteration in Python. For $N = 5, 50, 150$ plot the *normwise backward error*

$$\eta^{(n)} = \frac{\|\underline{r}^{(n)}\|_1}{\|\underline{\underline{A}}\|_1 \|\underline{\hat{x}}^{(n)}\|_1 + \|\underline{b}\|_1}, \quad \underline{r} = \underline{\underline{A}}\underline{\hat{x}}^{(n)} - \underline{b} \quad (7)$$

for $n = 0, 1, \dots, N$.

Problem 3. Floating point errors in linear solves

Floating point precision is limited such that even for an exact method such as LU factorization our result for $\underline{x}$ in $\underline{A}\underline{x} = \underline{b}$ will only be an approximation $\hat{\underline{x}}$.

A typical error measure for linear solvers is the *normwise backward error*

$$\eta = \frac{\|\underline{r}\|_1}{\|\underline{A}\|_1 \|\hat{\underline{x}}\|_1 + \|\underline{b}\|_1}, \quad \underline{r} = \underline{A}\hat{\underline{x}} - \underline{b} \quad (8)$$

For our tridiagonal solver from Problem 1, η will typically be around machine precision ϵ_{mach} , see Higham, 2002, Theorem 9.14.

In this task we will work with LU factorization with partial pivoting as one would use in practice, a minimal example is given in Code-Snippet 4.

We will consider two kinds of example matrices:

- random strictly diagonally dominant matrices $\underline{R}_N$ with

$$\forall i : |r_{ii}| > \sum_{j \neq i} |a_{ij}| \quad (9)$$

These can be generated by

```
1      import numpy as np
2      from scipy.linalg import lu_factor, lu_solve
3
4      # Define a small linear system A x = b
5      A = np.array([[3.0, 1.0],
6                    [1.0, 2.0]])
7      b = np.array([9.0, 8.0])
8
9      # LU factorization with partial pivoting
10     lu, piv = lu_factor(A)
11
12     # Solve the system using the LU factors
13     x = lu_solve((lu, piv), b)
14
15     print("Solution x:", x)
16
17     # Optional: check the result
18     print("A @ x:", A @ x)
```

Code-Snippet 4: Basic LU solve in Python.

- (a) Draw a random Gaussian matrix $\underline{\underline{G}}_N \in \mathbb{R}^{n \times n}$, $G_{ij} \sim \mathcal{N}(\mu = 0, \sigma^2 = 1)$.
- (b) Enforce strict diagonal dominance by

$$R_{ij} = \begin{cases} G_{ij}, & i \neq j, \\ \delta + \sum_{j \neq i} |G_{ij}|, & i = j \end{cases} \quad (10)$$

with margin $\delta = 0.1$.

- Wilkinson matrices

$$W_n = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 & 1 \\ -1 & 1 & 0 & \cdots & 0 & 1 \\ -1 & -1 & 1 & \ddots & \vdots & \vdots \\ \vdots & \vdots & \ddots & \ddots & 0 & 1 \\ -1 & -1 & \cdots & -1 & 1 & 1 \\ -1 & -1 & \cdots & -1 & -1 & 1 \end{bmatrix} \in \mathbb{R}^{n \times n}, \quad W_{ij} = \begin{cases} 1 & \text{if } i = j \text{ or } j = n \\ -1 & \text{if } i > j \\ 0 & \text{otherwise} \end{cases} \quad (11)$$

A Gaussian RHS $b_i \sim \mathcal{N}(\mu = 0, \sigma^2 = 1)$, $i = 1, \dots, N$ is used throughout this task.

- (a) Implement functions to generate $\underline{\underline{W}}_N$ and $\underline{\underline{R}}_N$.
- (b) Implement a function to calculate η .
- (c) For both $\underline{\underline{W}}_N$ and $\underline{\underline{R}}_N$ for $N = 5, 50, 100, 150$ calculate η . Plot η over N .
- (d) For $\underline{\underline{W}}_4$ go through Gaussian elimination with partial pivoting by hand. Explain the immense accumulation of floating point errors with increasing N .

Problem 4. (*Bonus*) Polynomial regression

Given the data

$$\begin{aligned} \underline{x} &= \begin{pmatrix} -3.00, -2.85, -2.70, -2.55, -2.40, -2.25, -2.10, -1.95, \\ -1.80, -1.65, -1.50, -1.35, -1.20, -1.05, -0.90, -0.75, \\ -0.60, -0.45, -0.30, -0.15, 0.00, 0.15, 0.30, 0.45, \\ 0.60, 0.75, 0.90, 1.05, 1.20, 1.35, 1.50, 1.65, \\ 1.80, 1.95, 2.10, 2.25, 2.40, 2.55, 2.70, 2.85, \end{pmatrix} \in \mathbb{R}^{40}, \\ \underline{y}^{\text{data}} &= \begin{pmatrix} -6.74, -6.01, -4.83, -4.37, -4.01, -2.94, -1.90, -1.56, \\ -1.92, -1.77, -1.07, -0.41, -1.30, 0.01, -0.28, 0.16 \\ 0.41, 0.64, 1.10, 1.48, 0.94, 1.70, 0.69, 1.17, \\ 1.41, 0.96, 0.48, 0.31, 0.46, 0.71, 0.01, 0.31, \\ 0.24, 0.50, 0.25, 0.24, -0.34, -0.15, 0.26, 0.57, \end{pmatrix} \in \mathbb{R}^{40} \end{aligned} \quad (12)$$

we would like to find parameters $\underline{\beta} \in \mathbb{R}^p$ in the relation

$$y_i(\underline{\beta}) = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \dots + \beta_{p-1} x_i^{p-1} \quad (13)$$

such that $\|\underline{y}^{\text{data}} - \underline{y}(\underline{\beta})\|_2^2$ is minimized. Note that $y_i(\underline{\beta})$ is linear in $\underline{\beta}$, polynomial regression is a linear regression.

Given the data matrix (here $N = 40$)

$$\underline{\underline{X}} = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^{p-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_N & x_N^2 & \cdots & x_N^{p-1} \end{bmatrix} \in \mathbb{R}^{N \times p} \quad (14)$$

The parameters of interest are given by

$$\hat{\underline{\beta}} = \underset{\underline{\beta}}{\operatorname{argmin}} \|\underline{y}^{\text{data}} - \underline{y}(\underline{\beta})\|_2^2 \iff \underline{\underline{X}}^T \underline{\underline{X}} \hat{\underline{\beta}} = \underline{\underline{X}}^T \underline{y}^{\text{data}} \quad (15)$$

i.e. a linear system.

- Show that $\underline{\underline{X}}^T \underline{\underline{X}}$ is invertible for $p < N$.
- Write Python functions to construct the data matrix $\underline{\underline{X}}$ and to evaluate $\underline{y}(\underline{\beta})$.
- Solve for $\underline{\beta}$ for $p = 1, 2, 4, 12$. Plot the results and the data. Describe what you observe.

References

Higham, Nicholas J. (2002). *Accuracy and Stability of Numerical Algorithms*. Second. Society for Industrial and Applied Mathematics. DOI: 10.1137/1.9780898718027. URL: <https://epubs.siam.org/doi/abs/10.1137/1.9780898718027>.